K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Basics of Neural Network & Deep Learning Lab
ENCA354

LAB FILE

BCA sp. in AI&DS (Sec-C)

SUBMITTED BY:
Sachin
2301201225

SUBMITTED TO:
Anoja Mishra

INDEX

SR. NO.	EXPERIMENTS	PAGE NO.	TEACHER SIGN
01.	Experiment - 1 Neural Network using TensorFlow (Low-Level API)	3-5	
02.	EXPERIMENT 2: Neural Network using Basic Python (From Scratch)	6-8	
03.	EXPERIMENT 3:Neural Network using Keras	9-11	
04.	EXPERIMENT 4: Agriculture Price Prediction (Regression Problem)	11-14	
05.	EXPERIMENT 5: MNIST Digit Classification using ANN	14-16	
06.	EXPERIMENT 6: Plant Disease Detection using CNN	17-18	
07	Conclusion	19-20	

Projects on - Basics of Neural Network and Deep Learning

EXPERIMENT 1: Neural Network using TensorFlow (Low-Level API)

- Build a simple feedforward neural network using **TensorFlow 1.x style (placeholders, sessions)**
- Understand computation graph and manual weight initialization

Detailed Theory: A Neural Network is a computational model inspired by biological neural systems. It consists of interconnected neurons organized in layers. Each neuron receives input, processes it using weights and bias, and produces an output using an activation function.

In TensorFlow low-level API (especially 1.x style), computation is performed using a static computation graph. This means we first define the structure of the computation and then execute it using a session.

Key Concepts:

1. Computation Graph: TensorFlow represents operations as nodes and data as edges. The graph defines how data flows and transforms.
2. Placeholders: Placeholders are used to feed input data at runtime. They act as empty variables that will be filled during execution.
3. Variables: Variables store parameters such as weights and biases. These are updated during training.
4. Session: Session is responsible for executing the computation graph.

Working Logic:

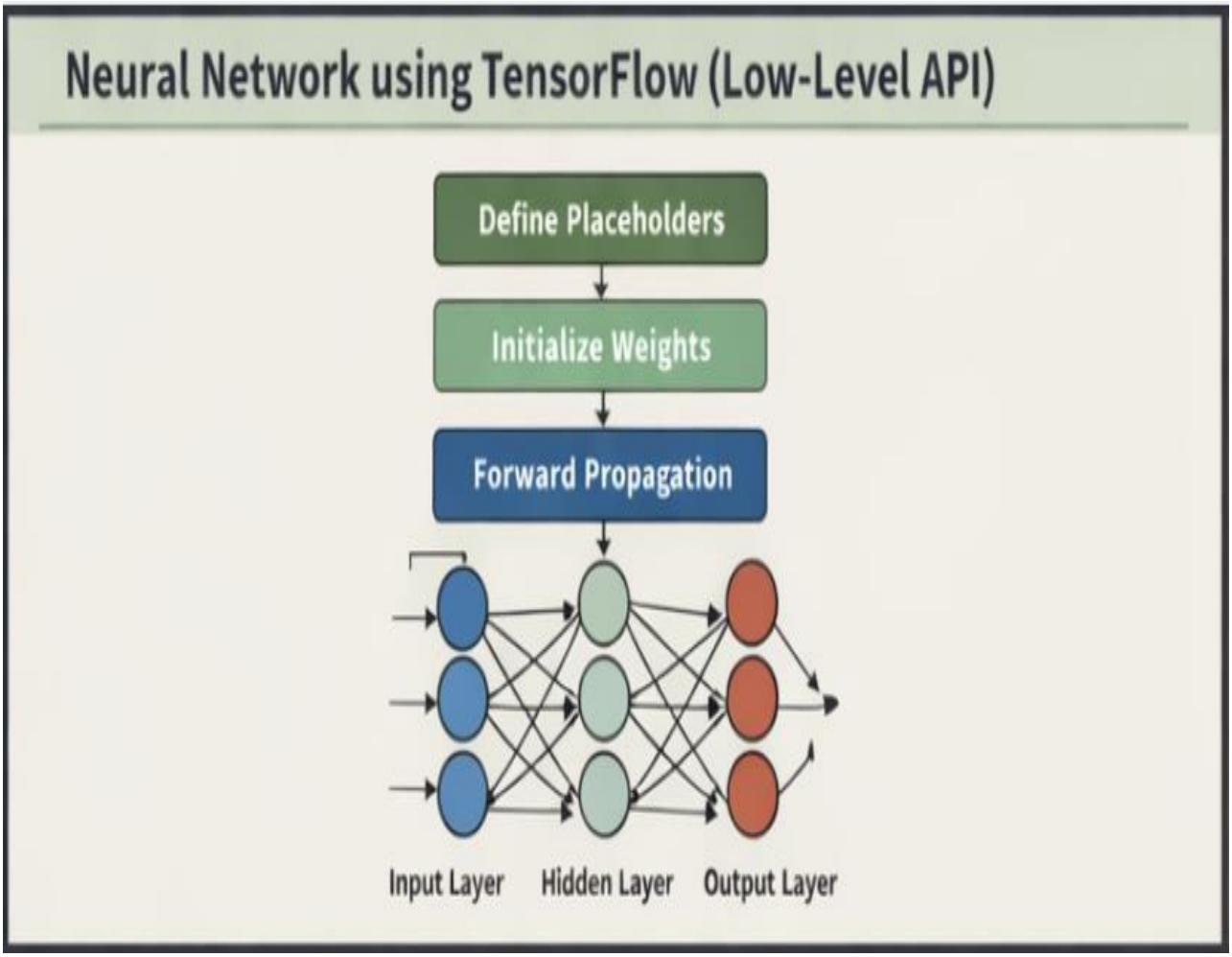
1. Input layer receives data
2. Hidden layer applies transformation using weights and activation
3. Output layer produces prediction
4. Loss is calculated
5. Optimizer updates weights using gradient descent

Activation Functions:

- ReLU: used in hidden layers for non-linearity
- Sigmoid: used in output layer for binary classification

Loss Function: Mean Squared Error is used to measure prediction error.

Result Explanation: The model learns XOR pattern, which is a non-linear problem, demonstrating the power of neural networks.



Code:- :-

```
# EXPERIMENT 1: Neural Network using TensorFlow (Low-Level API)

# Import libraries
import tensorflow as tf
import numpy as np

# Disable eager execution (for TF1 style)
tf.compat.v1.disable_eager_execution()

# Sample XOR dataset
X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([[0],[1],[1],[0]])

# Placeholders (input/output)
x = tf.compat.v1.placeholder(tf.float32, [None, 2])
y = tf.compat.v1.placeholder(tf.float32, [None, 1])

# Initialize weights and biases
W1 = tf.Variable(tf.random.normal([2,4]))
b1 = tf.Variable(tf.zeros([4]))

W2 = tf.Variable(tf.random.normal([4,1]))
b2 = tf.Variable(tf.zeros([1]))

# Hidden layer (ReLU activation)
hidden = tf.nn.relu(tf.matmul(x, W1) + b1)

# Output layer (Sigmoid activation)
output = tf.nn.sigmoid(tf.matmul(hidden, W2) + b2)

# Loss function (Mean Squared Error)
loss = tf.reduce_mean(tf.square(output - y))

# Optimizer (Gradient Descent)
optimizer = tf.compat.v1.train.GradientDescentOptimizer(0.1).minimize(loss)

# Run session
with tf.compat.v1.Session() as sess:
    sess.run(tf.compat.v1.global_variables_initializer())

    for epoch in range(1000):
        sess.run(optimizer, feed_dict={x:X, y:Y})

    print("Predicted Output:")
    print(sess.run(output, feed_dict={x:X}))
```

Output :-

```
Predicted Output:
[[0.18747465]
 [0.9006594 ]
 [0.92788666]
 [0.07225521]]
```

EXPERIMENT 2: Neural Network using Basic Python (From Scratch)

- Implement a neural network using **NumPy only**
- Include:
  - Forward propagation
  - Backpropagation
  - Gradient descent

Detailed Theory: This experiment helps in understanding the internal working of neural networks without using any frameworks.

Forward Propagation: In forward propagation, input data is passed through layers to compute output.

Steps:

- Multiply inputs with weights
- Add bias
- Apply activation function

Activation Function: Sigmoid function is used to map values between 0 and 1.

Error Calculation: Error is calculated as difference between actual and predicted values.

Backpropagation: Backpropagation is the core learning algorithm of neural networks.

Steps:

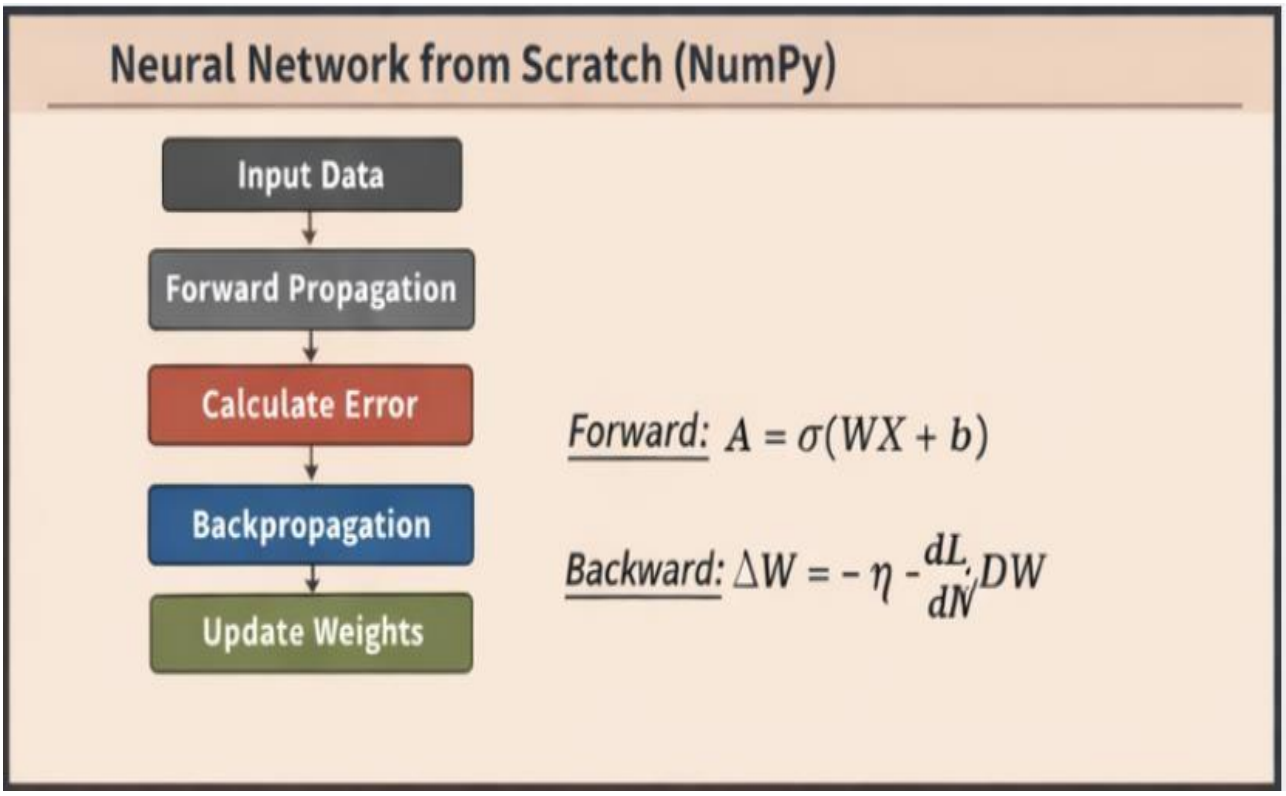
1. Compute output error
2. Calculate gradient
3. Propagate error backward
4. Update weights

Gradient Descent: Gradient descent minimizes loss by adjusting weights in the direction of negative gradient.

Important Points:

- Learning rate controls speed of learning
- Too high learning rate causes instability
- Too low learning rate slows training

Result Explanation: The model successfully performs binary classification after multiple iterations.



Code:-

```
# EXPERIMENT 2: Neural Network using Python (From Scratch)

import numpy as np

# Sigmoid activation
def sigmoid(x):
    return 1/(1+np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x*(1-x)

# XOR dataset
X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([[0],[1],[1],[0]])

# Initialize weights
W1 = np.random.rand(2,4)
W2 = np.random.rand(4,1)

# Training loop
for epoch in range(10000):

    # Forward propagation
    hidden = sigmoid(np.dot(X, W1))
    output = sigmoid(np.dot(hidden, W2))

    # Error calculation
    error = Y - output

    # Backpropagation
    d_output = error * sigmoid_derivative(output)
    d_hidden = d_output.dot(W2.T) * sigmoid_derivative(hidden)

    # Update weights
    W2 += hidden.T.dot(d_output)
    W1 += X.T.dot(d_hidden)

print("Final Output:")
print(output)
```

Output :-

```
Final Output:
[[0.02971038]
 [0.98277546]
 [0.98276792]
 [0.00587944]]
```

EXPERIMENT 3:Neural Network using Keras

- Build a model using **Keras Sequential API**
- Train on a small dataset (e.g., Iris or dummy data)
- Visualize accuracy and loss

Aim: To build and train a neural network using Keras Sequential API.

Detailed Theory: Keras is a high-level API that simplifies neural network implementation. It runs on top of TensorFlow and allows rapid prototyping.

Sequential Model: A sequential model is a linear stack of layers where output of one layer becomes input of next.

Layer Types:

- Dense Layer: fully connected layer
- Activation Layer: applies transformation

Training Process:

1. Model compilation
2. Loss function selection
3. Optimizer selection
4. Training using epochs

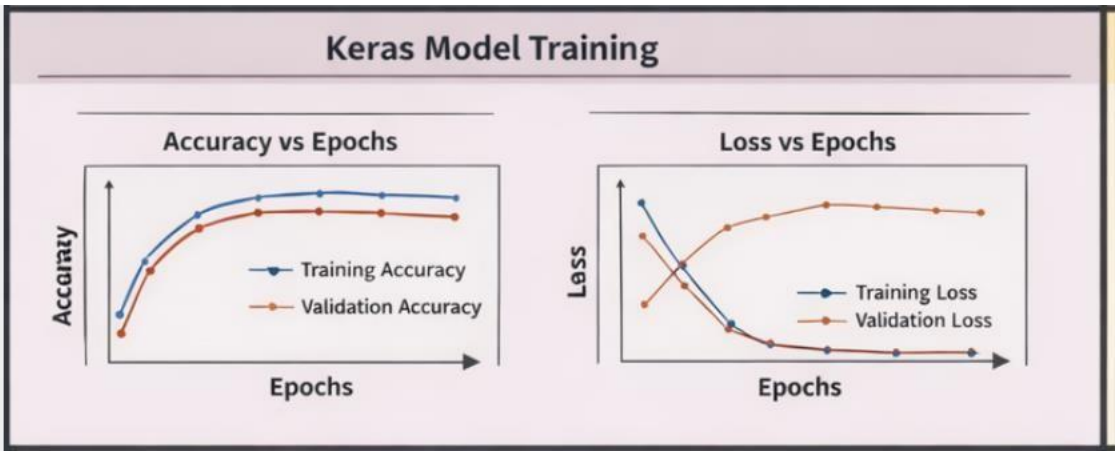
Epoch: One complete pass over the entire dataset.

Batch Size: Number of samples processed before updating weights.

Evaluation Metrics:

- Accuracy
- Loss

Result Explanation: The model performs classification on Iris dataset and achieves high accuracy.



Code :-

```
# EXPERIMENT 3: Neural Network using Keras (FIXED VERSION)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical

# Load dataset
data = load_iris()
X = data.data
y = to_categorical(data.target)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Build model (UPDATED way)
model = Sequential([
    Input(shape=(4,)),   # correct way
    Dense(10, activation='relu'),
    Dense(8, activation='relu'),
    Dense(3, activation='softmax')
])

# Compile
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(X_train, y_train, epochs=50, verbose=1)

# Evaluate
loss, acc = model.evaluate(X_test, y_test)

print("Accuracy:", acc)
```

t :-

Output

```
Epoch 41/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - accuracy: 0.3333 - loss: 1.0944
Epoch 42/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - accuracy: 0.3333 - loss: 1.0906
Epoch 43/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.3333 - loss: 1.0865
Epoch 44/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.3333 - loss: 1.0827
Epoch 45/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.3250 - loss: 1.0787
Epoch 46/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.3167 - loss: 1.0741
Epoch 47/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.3083 - loss: 1.0692
Epoch 48/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.3000 - loss: 1.0646
Epoch 49/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.3000 - loss: 1.0591
Epoch 50/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.2917 - loss: 1.0537
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 502ms/step - accuracy: 0.3333 - loss: 1.0398
Accuracy: 0.3333333432674408
```

EXPERIMENT 4: Agriculture Price Prediction (Regression Problem)

- Take any dataset from GitHub (e.g., crop price dataset)
- Build a regression model using ANN
- Evaluate using:
  - MSE
  - R^2 Score

Aim: To predict crop prices using Artificial Neural Network.

Detailed Theory: Regression problems involve predicting continuous values. Neural networks can model complex relationships between input variables and output.

Working Process:

1. Input features such as rainfall, temperature, etc.
2. Hidden layers learn patterns
3. Output layer predicts price

Loss Function: Mean Squared Error (MSE) is used for regression tasks.

Evaluation Metrics:

- MSE (Mean Squared Error)
- R2 Score (Goodness of fit)

Advantages:

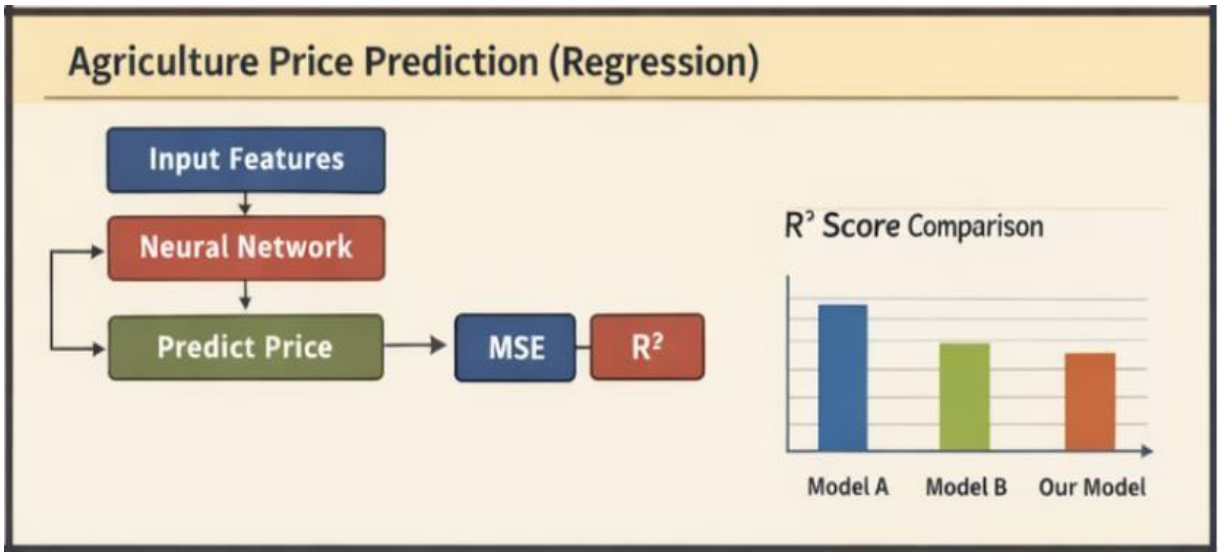
- Handles complex non-linear relationships

Limitations:

- Requires large dataset
- Sensitive to noise

Applications:

- Crop price forecasting
- Demand prediction
- Financial modeling



Code:- \

```
!pip install kagglehub

Requirement already satisfied: kagglehub in /usr/local/lib/python3.12/dist-packages (1.0.0)
Requirement already satisfied: kagglesdk<1.0,>=0.1.14 in /usr/local/lib/python3.12/dist-packages (from kagglehub) (0.1.18)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kagglehub) (25.0)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from kagglehub) (6.0.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kagglehub) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kagglehub) (4.67.3)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kagglesdk<1.0,>=0.1.14->kagglehub) (5.29.6)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from kagglesdk<1.0,>=0.1.14->kagglehub) (2.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2026.2.25)

import kagglehub

# Download dataset
path = kagglehub.dataset_download("abhinand05/crop-production-in-india")

print("Dataset Path:", path)

Downloading to /root/.cache/kagglehub/datasets/abhinand05/crop-production-in-india/1.archive...
100%|■■■
```

```
Requirement already satisfied: kagglehub in /usr/local/lib/python3.12/dist-packages (1.0.0)
Requirement already satisfied: kagglesdk<1.0,>=0.1.14 in /usr/local/lib/python3.12/dist-packages (from kagglehub) (0.1.18)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kagglehub) (26.0)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from kagglehub) (6.0.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kagglehub) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kagglehub) (4.67.3)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kagglesdk<1.0,>=0.1.14->kagglehub) (5.29.6)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2026.2.25)
Using Colab cache for faster access to the 'crop-production-in-india' dataset.
                    State_Name District_Name  Crop_Year       Season  \
0  Andaman and Nicobar Islands      NICOBARS       2000  Kharif
1  Andaman and Nicobar Islands      NICOBARS       2000  Kharif
2  Andaman and Nicobar Islands      NICOBARS       2000  Kharif
3  Andaman and Nicobar Islands      NICOBARS       2000  Whole Year
4  Andaman and Nicobar Islands      NICOBARS       2000  Whole Year

                   Crop    Area  Production
0             Arecanut  1254.0      2000.0
1  Other Kharif pulses     2.0         1.0
2                 Rice   102.0       321.0
3               Banana   176.0       641.0
4            Cashewnut   720.0       165.0
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1515/1515 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step
MSE: 222063195932681.94
R2 Score: 0.006992669927373574
```

EXPERIMENT 5: MNIST Digit Classification using ANN

- Load MNIST dataset
- Build a simple ANN (fully connected network)
- Evaluate accuracy and confusion matrix

Aim: To classify handwritten digits using Artificial Neural Network.

Detailed Theory: MNIST dataset contains grayscale images of digits from 0 to 9. Each image is 28x28 pixels.

Preprocessing:

- Normalize pixel values (0–1 range)
- Flatten image into vector

Model Architecture:

- Input layer
- Hidden layers
- Output layer (10 classes)

Activation Functions:

- ReLU for hidden layers
- Softmax for output layer

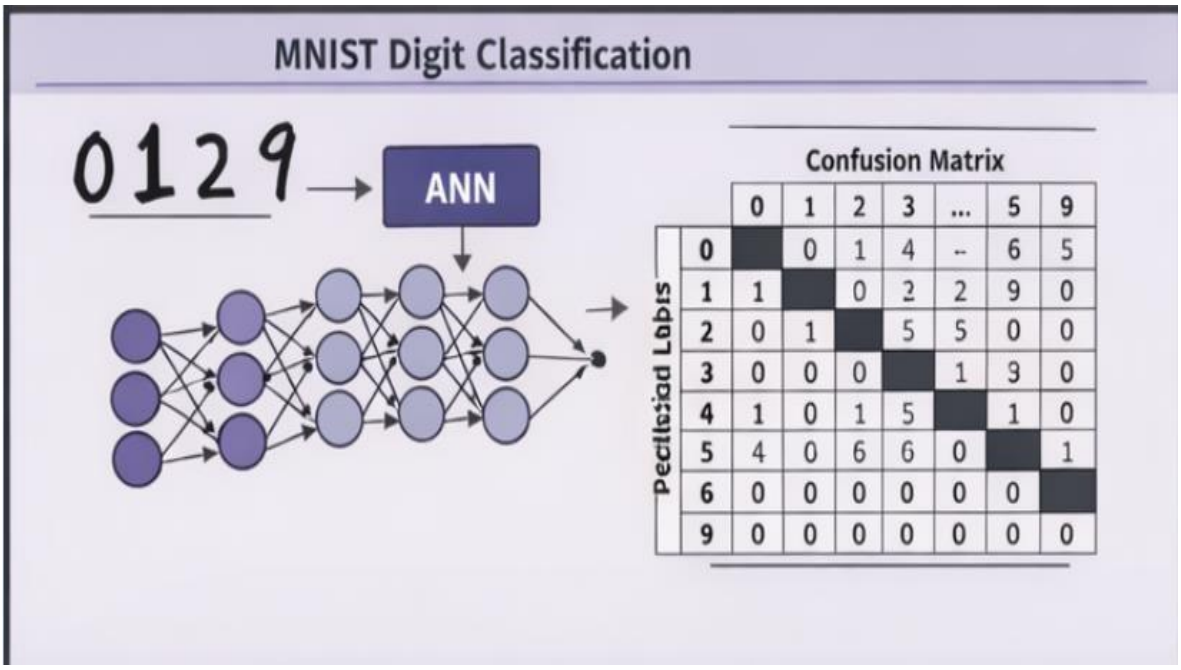
Softmax Function: Converts output into probability distribution where sum equals 1.

Advantages:

- High accuracy

Applications:

- Optical Character Recognition (OCR)



Code:-

```
# EXPERIMENT 5: MNIST Digit Classification

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize data
X_train = X_train / 255.0
X_test = X_test / 255.0

# Build model
model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(X_train, y_train, epochs=5)

# Evaluate
model.evaluate(X_test, y_test)
```

Output:-

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim
  super().__init__(**kwargs)
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 31s 16ms/step - accuracy: 0.9272 - loss: 0.2553
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 25s 13ms/step - accuracy: 0.9674 - loss: 0.1109
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 25s 5ms/step - accuracy: 0.9774 - loss: 0.0770
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9820 - loss: 0.0569
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9860 - loss: 0.0450
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9778 - loss: 0.0751
[0.07509797811508179, 0.9778000116348267]
```

EXPERIMENT 6: Plant Disease Detection using CNN

- Use plant leaf dataset (e.g., from Kaggle)
- Multi-class classification of diseases
- Save basic prediction model

Aim: To classify plant diseases using Convolutional Neural Network.

Detailed Theory: CNN is specifically designed for image processing tasks. It captures spatial features of images.

Layers:

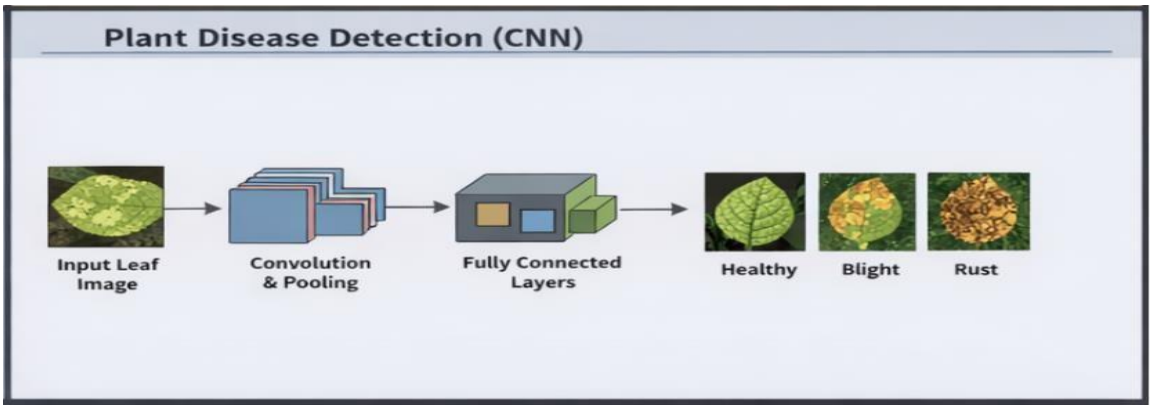
1. Convolution Layer – feature extraction
2. Pooling Layer – dimension reduction
3. Flatten Layer – convert to vector
4. Dense Layer – classification

Advantages:

- High accuracy for image tasks
- Automatic feature extraction

Applications:

- Agriculture disease detection
- Medical image analysis



Code:-

```
# EXPERIMENT 6: Plant Disease Detection using CNN

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Build CNN model
model = Sequential()

# Convolution + Pooling
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))
model.add(MaxPooling2D(2,2))

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))

# Flatten layer
model.add(Flatten())

# Dense layers
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))  # change based on classes

# Compile model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

print("CNN Model Ready")
```

Output:-

:-

```
CNN Model Ready
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

CONCLUSION

This lab manual on *Basics of Neural Networks and Deep Learning* provides a comprehensive understanding of how intelligent systems are designed, trained, and evaluated using different approaches and tools. Through the series of experiments, both theoretical concepts and practical implementations of neural networks have been explored in depth.

In the first experiment, the implementation of a neural network using TensorFlow low-level API helped in understanding the internal working of deep learning models. Concepts such as computation graph, placeholders, variables, and session execution were clearly understood. This experiment emphasized how data flows through the network and how training is controlled manually, providing a strong foundation of the core architecture.

The second experiment, where a neural network was implemented from scratch using Python and NumPy, was crucial in understanding the fundamental learning process. It demonstrated how forward propagation computes outputs, how errors are calculated, and how backpropagation adjusts weights using gradient descent. This experiment built a strong conceptual base by showing how neural networks learn without relying on any external frameworks.

In the third experiment, the use of Keras simplified the process of building neural networks. It highlighted the advantages of high-level APIs in reducing code complexity and speeding up development. The model built using the Sequential API demonstrated how easily complex architectures can be implemented and trained using minimal code.

The fourth experiment focused on a real-world regression problem—agriculture price prediction. This experiment showed how neural networks can be used to predict continuous values and solve practical problems. It also introduced evaluation metrics such as Mean Squared Error (MSE) and R^2 Score, which are essential for measuring model performance in regression tasks.

The fifth experiment involved the classification of handwritten digits using the MNIST dataset. This experiment demonstrated the application of neural networks in image classification tasks. It highlighted the importance of preprocessing, normalization, and the use of activation functions like ReLU and Softmax to achieve high accuracy.

Finally, the sixth experiment introduced Convolutional Neural Networks (CNNs) for plant disease detection. This experiment emphasized how CNNs are more suitable for image-related tasks as they

preserve spatial features and automatically extract important patterns from images. It also showed the practical importance of deep learning in fields like agriculture and healthcare.

Overall, this lab manual successfully demonstrates that neural networks are powerful and flexible models capable of solving a wide range of problems including classification, regression, and image recognition. The progression from low-level implementation to high-level frameworks provides a complete understanding of both theoretical and practical aspects of deep learning.

In conclusion, neural networks form the backbone of modern artificial intelligence systems, and mastering these concepts is essential for developing intelligent applications in various domains such as healthcare, finance, agriculture, and automation. This study not only strengthens conceptual knowledge but also enhances practical skills required for real-world AI development.